Functions

CSE 101 Section 03

Introduction to Algorithms and Computing

Fall 2023

© Royal Military College of Canada

Motivating Example

```
print( "Program starts" )

print( "Hi Elisa" )
print( "Nice to see you again!" )
print( "Enjoy this lecture!" )

# some lines of codes
the_answer = 42

print( "Hi Imad" )
print( "Nice to see you again!" )
print( "Enjoy this lecture!" )

width, length= 3 , 4
area = width*length

print("Hi Logan")
print("Nice to see you again!")
print("Enjoy this lecture!")
```

Output

```
Program starts
Hi Elisa
Nice to see you again!
Enjoy this lecture!
Hi Imad
Nice to see you again!
Enjoy this lecture!
Hi Logan
Nice to see you again!
Enjoy this lecture!
```

Motivating Example

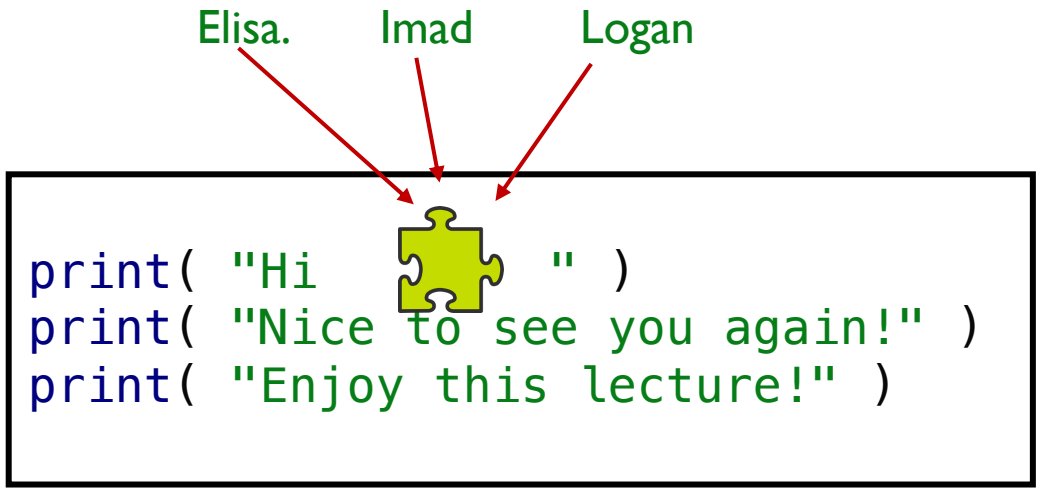
```
print( "Program starts" )

print( "Hi Elisa" )
print( "Nice to see you again!" )
print( "Enjoy this lecture!" )

# some lines of codes
the_answer = 42

print( "Hi Imad" )
print( "Nice to see you again!" )
print( "Enjoy this lecture!" )

width, length= 3 , 4
area = width*length

print("Hi Logan")
print("Nice to see you again!")
print("Enjoy this lecture!")
```

- You can see in the code that we are greeting three people.
- Every time we use three print calls which are nearly the same, with the name being the only difference.

→ Redundant code

Motivating Example: how to fix it?

```
print( "Program starts" )

print( "Hi Elisa" )
print( "Nice to see you again!" )
print( "Enjoy this lecture!" )

# some lines of codes
the_answer = 42

print( "Hi Imad" )
print( "Nice to see you again!" )
print( "Enjoy this lecture!" )

width, length= 3 , 4
area = width*length

print("Hi Logan")
print("Nice to see you again!")
print("Enjoy this lecture!")
```



How to do this in Python? Functions!

```
print( "Program starts" )

def greet(name):
    print("Hi " + name)
    print("Nice to see you again!")
    print("Enjoy this lecture!")

print("Program starts")

greet("Elisa")
# some lines of codes
the_answer = 42

greet("Imad")

width, length = 3, 4
area = width * length

greet("Logan")
```

Function definition: this is what the function does

Function call: We call the function with the piece of information that it needs, in this case: the name

How to do this in Python? Functions!

```
print( "Program starts" )

def greet(name):
    print("Hi " + name)
    print("Nice to see you again!")
    print("Enjoy this lecture!")

print("Program starts")

greet("Elisa")
# some lines of codes
the_answer = 42

greet("Imad")

width, length = 3, 4
area = width * length

greet("Logan")
```

Function definition: this is what the function does

Function call: We call the function with the piece of information that it needs, in this case: the name

General Syntax of a Python function

```
def functionName():
    statement
    statement
```

- The name of the function should reflect what it does. Same naming rules as for variables.

- Examples: `findAverage()`
  `printReport()`

Example

```
def f(x, y):
    z = 2 * (x + y)
    return z

print("Program starts!")
a = 3
res1 = f(a, 2+a)
print("Result of function call:", res1)
a = 4
b = 7
res2 = f(a, b)
print("Result of function call:", res2)
```

Example

function name

function definition

```
def f(x, y):
    z = 2 * (x + y)
    return z

print("Program starts!")
a = 3
res1 = f(a, 2+a)
print("Result of function call:", res1)
a = 4
b = 7
res2 = f(a, b)
print("Result of function call:", res2)
```

function call 1

function call 2

Input to the function

Output of the function

Functions in Python

- A grouping of statements together to perform some functionality.

Benefits

- Readability
- Reusability
- Modularity (i.e., breaking program into smaller tasks)

- A function may have **0 or more parameters** (input)
- A function may have **0 or more returned values** (output).

Local and Global Variables in Functions

- **Local variable: variable that is assigned a value inside a function**
  - Belongs to the function in which it was created
  - Only statements inside that function can access it, error will occur if another function tries to access the variable

```
def myFun():
    x = 6
```

```
def myFun():
    x = 6
print(x)   #error!!!!
```

Local Variables (cont'd.)

- **<u>Scope</u>: the part of a program in which a variable may be accessed**
- For local variable: function in which variable was created

- **Different functions may have local variables with the same name**
- Each function does not see the other function's local variables, so no confusion

```
def myFun():
    x = 6
    print(x)
def yourFun():
    x = 25
    print(x)

myFun()
yourFun()
```

Input to the function – aka parameters

```
def f(x, y):
    z = 2 * (x + y)
    return z
```

The function f takes two parameters x and y

```
a = 2
b = 3
f(a, b)
```

To use the function f, we call it by typing the name of the function followed by brackets that enclose the parameters that the function f expects.

Output of the function – aka returned values

```
def f(x, y):
    z = 2 * (x + y)
    return z
```

The function f returns one value z

```
a = 2
b = 3
f(a, b)
```

Since variable z is local to function f. It is not accessible here.

```
result = f(a, b)
```

Therefore, to access/use the value in z, we can store the returned value of f in a new variable.

Another example

```
def fahrenheit(T_in_celsius):
    """ returns the temperature in degrees Fahrenheit """
    T_in_F = (T_in_celsius * 9 / 5) + 32
    return T_in_F

for t in (22.6, 25.8, 27.3, 29.8):
    print(t, ": ", fahrenheit(t))
```

And another one

```
def singHappyBD():
    print("Happy birthday to you")
    print("Happy birthday to you")
    print("Happy birthday to you, Princess")
    print("Happy birthday to you!")
def main():
    singHappyBD()
    singHappyBD()

main()
```

Practice

- Modify the `singHappyBD()` to take one parameter, i.e. the name of the birthday person and use the name in the song.

```python
def function1():
    x = 5
    return x
def function2():
    x = "elephant"
def function3():
    x = 17
    return x


x = function1()
print(x)

x = function2()
print(x)

x = function3()
print(x)
```

Because variables are defined inside functions, we can have many with the same names. They are NOT the same variables!

```python
def isValidInput (value):
    ''' Checks to ensure that the value is even and between 0 and 1000'''
    if value % 2 == 0 and value >= 0 and value <= 1000:
        return True
    return False

def getInput():
    num = -1
    while not(isValidInput(num)): #not valid input
        num = int(input("Enter an even number up to 1000"))
    #now I have valid input...so continue
```

How to comment functions

```
def myFunction(a, b):
    """
    This function adds the values of the two parameters, a and b and returns
the result.
    Parameters:  a, b - integer values representing some interesting things
    Return Value - the sum of a and b - an integer
    """
```